

also assume an *index.php* file in */test*, accessible at www.example.com/test/index.php. If *index.php* has the above vulnerable PHP code in it, we should be able to visit www.example.com/test/index.php?page=../test/test.php and thus execute *test.php*. (The *../* is a directory transversal, which goes up one directory.)

If this website was hosted on a *NIX server, then we might be able to do a directory transversal to the *passwd* file and others, like (assuming *index.php* is in */var/www/test*): www.example.com/test/index.php?page=../../etc/passwd. (The effort here is add *../* until you get the desired directory, if you don't know where *test.php* is located on the server.) Apart from */etc/passwd*, attackers also try to grab */etc/shadow*, */etc/group*, */etc/security/group*, */etc/security/passwd*, */etc/security/user*, */etc/security/environ*, */etc/security/limits*, and */usr/lib/security/mkuser.default*.

Most often, as a security solution, developers just append a *“.php”* to the end of the URL, so that an attacker's request for */etc/passwd*, as shown above, would change the request to */etc/passwd.php*. This is a non-existent file. However, to work around this protection, the attacker could just use a *nullbyte* — i.e., add *%00* to the end of the URL before sending: */etc/passwd%00*. The *nullbyte* causes anything appended after it to be ignored, thus forcing the original request for */etc/passwd*.

Apache log poisoning with LFI

The above LFI example lets an attacker read files on the server. But there are also ways for attackers to execute the desired code—mainly by exploiting Apache's log files. Apache normally uses two log files: *access_log*, which contains all valid requests to the Web server, and *error_log*, which contains error messages. Apache lets one customise the verbosity of these log files—administrators can choose if they ought to contain information about the browser, OS, referring URLs from the request. Since attackers can control what information they submit here, they can simply change their OS or browser information to be valid PHP code, instead of benign client information.

Since Apache supports virtual hosts, this means that it can handle multiple domains on the same IP, and since the administrator can specify the name and location of the log file, it is very hard to know where the Apache logs are stored. However, knowing the location is important for a successful attack. Here are some of the possible/common locations where Apache logs might be stored (they could also be named *access.log* and *error.log*):

```
/etc/httpd/logs/, /opt/lampp/logs/, /usr/local/apache/
log/, /usr/local/apache/logs/, /usr/local/etc/httpd/logs/, /usr/
local/www/logs/httpd_log/, /var/apache/logs/,
/var/log/apache/, /var/log/apache-ssl/, /var/log/httpd/,
/var/log/httpsd/ssl.access_log (on *NIX systems) and C:\
apache\logs/, C:\Program Files\Apache Group\Apache\
```

logs, C:\program files\wamp\apache2\logs, C:\wamp\apache2\logs, C:\wamp\logs, C:\xampp\apache\logs\ on Windows systems.

Now, first, let's take the case of an attacker exploiting the *error_log* file. On an LFI-vulnerable server, the attackers might execute the following URL:

```
http://www.example.com/<?php==$_GET;@chdir($_['x']);echo@
system($_['y'])?>
```

Next, they will try to reach the above inserted PHP code at:

```
http://www.example.com/test/index.php?page=/var/log/apache/logs/
error_log%00&x=/&y=uname
```

This will result in displaying the server's information to the attackers. What has just happened is that the *<?PHP \$=\$_GET;@chdir(\$_['x']);echo@system(\$_['y'])?>* does not exist, so the PHP code is stored in *error_log* and then is executed when the second URL is called.

Now, let's look at the second case -- of exploiting *access_log*. Here, the attackers spoof their User-Agent and make it PHP code (like in the previous example), using a Firefox add-on called 'User Agent Switcher'. To log an entry with this changed User Agent string, they simply visit <http://www.example.com>. Then, they force *access_log* to be included, to execute the code (the attack URL is similar to the above, only the log file name changes).

Time for security

Most of the security for LFI attacks can be effectively handled with security solutions explained in Part 7 (directory-traversal attacks). However, apart from that, the following measures should also be applied:

1. In general, the best way to avoid script-injection vulnerabilities is to not pass user-supplied input, or data derived from it, into any dynamic execution or include functions.
2. If this is unavoidable for some reason, then the relevant input should be strictly validated to prevent any attack occurring.
3. If possible, use a white-list of known good values (such as a list of all languages or locations supported by the application), and reject any input that does not appear in this list. Failing that, check the characters used within the input against a set known to be harmless, such as alphanumeric characters, excluding white-space.

Tools of the secure trade

- **Lfimap** (<http://lfimap.googlecode.com/files/lfimap-1.4.8.tar.gz>) is a Python script that challenges Web applications against Local File Inclusions (LFI) attacks. It is based on a set of directories that are injected in detected parameters.